

문제 4 디지털포렌식

SafeTensors low-nibble steganography

CRYPTO{G00D_J0B!_y0u_f0und_7h3_h1dd3n_s3cr37_1n_LLM}

1 최종 재현과 입력 무결성

제출 코드 solve.py는 Python 표준 라이브러리만 사용한다. 공식 TinyLlama ZIP에서 SafeTensors의 목표 embedding row만 순차적으로 읽고, F32 원시 byte의 low nibble을 결합해 FLAG를 출력한다.

```
python3 solve.py TinyLlama-1.1B-Chat-v1.0.zip \
--verify-sha256
```

입력	문제 PDF의 SHA-256
TinyLlama-1.1B-Chat-v1.0.zip	144155ad4b55ecf4e14f08457d4d8874ef656ea69e29632cc55bb97057269fa7
server.zip	1a0ecbdcdb1ed4e3e51069643690659002612fbccc5a7a27919df17b7ab49dd5c
내부 server.log	d7e3c1fb4c94754c80cedddb2a6caa0fb7f2aa6a3344bcb9a25e132d1f4cd5

--verify-sha256은 모델 ZIP을 8MiB씩 읽어 첫 번째 값을 직접 확인한다. 모델 전체나 tensor 전체를 메모리에 올리지는 않는다.

2 로그 분석에서 얻은 단서

17GB server.log를 ZIP member에서 한 줄씩 읽었다. 각 chat record의 q1, a1은 부족한 base64 =만 보충해 decode한 뒤 알파벳 Caesar +11을 적용했다. 복호문에 secret, square, flag, crypto, 4 bit, nibble이 있는 record만 후보로 남겼다.

```
model: TinyLlama/TinyLlama-1.1B-Chat-v1.0
chat_id: chat-000005
Q: What secret does the model have?
A: The LLM knows the secret. Trust me.
"square" for open sesame!!!
```

복호문 끝의 l은 padding이라는 근거가 없으므로 임의로 삭제하지 않았다. square 입력에 기록된 Wouldn't about 4 bits be enough?라는 응답은 low 4-bit 채널을 조사하게 한 가설이었다. 당시 generation 환경은 보존되지 않았으므로 이 응답의 결정론적 재현을 최종 공격의 전제로 삼지 않았다.

3 F32 tensor blind discovery

SafeTensors의 F32 하나는 little-endian 4 byte다. 각 값의 첫 byte에서

$$\text{nibble} = \text{raw_f32_bytes}[0] \& 0x0f$$

를 취했다. 모든 F32 tensor를 8MiB chunk로 순차 처리하면서 비영 nibble 수와 nibble-pair ASCII preview를 비교했다. 가장 두드러진 후보는 다음과 같았다.

```
tensor = model.embed_tokens.weight
row     = 0
non-zero low nibbles = 345
```

로그 증거 보존, 모든 tensor 순위화와 방어적 형식 검증은 분석용 완전판에 남기고, 제출 solve.py에는 최종 추출식을 사람이 바로 확인할 수 있도록 확정된 tensor와 row 경로만 남겼다.

완전판은 후보마다 1-based line number, 압축 해제 로그의 byte offset, line/record SHA-256, 일치 검색어, 복호 질문·답변, raw와 손실 없는 base64를 log-evidence.jsonl에 즉시 기록한다. 끝에는 전체 line·record·후보·decode 오류 수와 스트리밍 계산한 내부 로그 SHA-256을 남긴다. 논리 line은 기본 1MiB로 제한하고 더 긴 line은 작은 조각으로 끝까지 해시에 포함하되 정규식 분석에서 제외하므로 메모리는 입력 전체 크기가 아니라 chunk와 최대 line에 의해 제한된다.

4 payload 추출

SafeTensors의 data_offsets는 파일 절대 위치가 아니라 JSON header 직후 data buffer 기준이다. 제출 코드는 다음 순서로 동작한다.

1. ZIP 안의 유일한 .safetensors member를 연다.
2. 앞 8바이트의 little-endian header 길이와 JSON을 읽는다.
3. model.embed_tokens.weight가 2차원 F32인지 확인한다.
4. shape와 data_offsets로 첫 행만 순차 읽는다.
5. 각 F32의 첫 byte에서 low nibble을 취한다.
6. 두 nibble을 $(high \ll 4) | low$ 로 결합한다.
7. 복원 byte열에서 CRYPTO{...}를 찾아 출력한다.

분석용 parser는 header 길이와 JSON padding·duplicate key, dtype/shape, tensor byte 수, sub-byte 정렬, range hole/overlap과 row 경계를 먼저 검사한다. 정렬한 tensor range가 data buffer 전체를 빈틈과 중복 없이 덮지 않으면 fail-closed한다.

중간의 0 nibble이나 0 byte를 삭제하지 않으며, float로 변환했다가 다시 직렬화하지도 않는다. 복원 payload는 다음과 같다.

```
"The lighthouse opens at midnight. Meet me at the bar."
```

```
Well done, Agent H.
```

```
You traced the secBet into the model itself.
```

```
The flag is:
```

```
CRYPTO{G00D_J0B!_y0u_f0und_7h3_h1dd3n_s3cr37_1n_LLM}
```

secBet과 open sesame은 원 기록의 철자를 그대로 적었다. 코드는 복원된 원시 byte열에서 FLAG를 실제로 찾아야만 성공한다.

5 합성 검증과 재현 범위

```
python3 -m unittest -v \
  solutions.test_solve_04_digital_forensics
```

작은 합성 ZIP 회귀는 discovery/extract 왕복, nibble과 chunk 경계, empty/scalar tensor, 잘못된 padding·duplicate key·hole·overlap·sub-byte 범위 거부, 초장문 로그의 bounded skip과 offset/hash/raw evidence를 검사한다. 이는 parser와 스트리밍 경계 검증이며 공식 payload를 다시 측정했다는 주장은 아니다.

대형 TinyLlama ZIP과 약 17GB server.zip은 제출물에 중복 포함하지 않는다. 현재 문서 검토 환경에도 두 공식 대상 입력이 없어 기존 분석의 chat-000005, 비영 nibble 수 345와 payload를 새로 측정했다고 주장하지 않는다. 공식 모델 ZIP이 주어지면 제출 코드는 archive SHA-256, target tensor 경계와 FLAG를 다시 검증한다.

blind discovery부터 재검토하려면 공식 입력으로 생성한 log-evidence.jsonl과 모든 F32 tensor 통계인 discovery.json을 함께 보존해야 한다.

6 참고 자료

- [SafeTensors 공식 포맷 명세](#): header, 상대 data_offsets, little-endian row-major data와 검증 규칙.
- [TinyLlama-1.1B-Chat-v1.0 공식 모델 카드](#): 로그의 모델 식별자.
- Zhang et al., [TinyLlama: An Open-Source Small Language Model](#): 대상 모델군의 배경.